Project Report: PromptForge v2.0

A Multi-Agent AI System for Automated Prompt Engineering

Title Page

A Project Report on "PromptForge"

Submitted in the partial fulfillment of the requirements in the 8th semester of **Bachelor of Technology** in **Computer Science [Specialization]**

For **Course Name:** CAPSTONE PROJECT **Course Code:** 10ABTEC22851

SUBMITTED BY Student Name 1 : Register Number Student Name 2 : Register Number Student Name 3 : Register Number Student Name 4 : Register Number

APRIL 2026

Dr. APJ Abdul Kalam School of Engineering BENGALURU, KARNATAKA, INDIA – 560049

Abstract

The contemporary higher education landscape faces a significant challenge in ensuring that graduates successfully transition into professional roles that align with their skill sets. Traditional placement processes are often fragmented, relying on manual screening and subjective evaluations that fail to account for the complex, multi-dimensional nature of modern employability. This report proposes a comprehensive, data-driven framework designed to enable guaranteed student placement through the systematic application of machine learning, natural language processing (NLP), and predictive analytics. The framework integrates several critical modules: an automated skill evaluation system, a semantic job recommendation engine, an AI-powered resume tailoring tool, a multi-modal interview preparation suite, and a quantitative salary negotiation guide. By leveraging advanced algorithms such as Random Forests, XGBoost, and Sentence-BERT (S-BERT), the proposed system achieves high predictive accuracy—up to 91% for placement outcomes—and provides personalized interventions for students at risk of underperformance. The framework transcends simple keyword matching by employing dense vector representations to capture the semantic nuances of resumes and job descriptions. Furthermore, the system incorporates behavioral analytics to provide real-time feedback on interview performance, addressing non-verbal cues and emotional tone. This holistic approach is grounded in economic mechanism design and matching theory, ensuring that the placement process is both efficient and equitable. Experimental results suggest that the framework not only increases the likelihood of securing an initial offer but also optimizes the final compensation package through data-driven leverage forecasting. This research contributes to the field of educational technology by offering a scalable, end-to-end infrastructure for professional success in the digital age.

Keywords: Machine Learning, Student Placement Prediction, Natural Language Processing, Resume Optimization, Skill Gap Analysis, Employability Analytics.

1. Introduction

1.1 Purpose of Project

The primary purpose of the PromptForge project is to address the ambiguity and imprecision inherent in human-AI interactions. It transforms vague, high-level user prompts into detailed, structured, and high-performance instructions suitable for execution by large language models (LLMs). This automated "prompt engineering" saves time, improves output quality, and makes advanced AI more accessible to users without specialized skills.

1.2 Problem Definition

LLMs are powerful but sensitive to the quality of their input prompts. A vague prompt like "write a story about a robot" often yields generic or undesirable results. The process of refining this into a detailed prompt that specifies tone, style, perspective, constraints, and format is a manual, iterative, and time-consuming task. PromptForge is designed to automate this entire refinement process.

1.3 Scope of Project

The project encompasses a complete, standalone system with the following key features:

- **Multi-Agent Swarm:** Utilizes a swarm of specialized AI agents orchestrated by LangGraph for prompt analysis and engineering.
- **API-first Design:** Exposes all functionality through a well-documented FastAPI backend, featuring robust middleware for security and monitoring.
- **Conversational Memory & Caching:** Employs Redis for performance caching and Supabase (with pgvector) to learn user preferences and style over time.
- **Observability:** Integrated with Sentry for error tracking, Langsmith for LLM tracing, and optionally OpenTelemetry for infrastructure tracing.
- **Containerized Deployment:** Fully containerized with Docker for easy and reproducible deployment across different environments.

1.4 Existing System

The existing "system" for most users is a manual, trial-and-error process of prompt refinement. Users type a prompt into an LLM interface, evaluate the output, and manually tweak the prompt to get closer to their desired result. This relies heavily on the user's own skill, patience, and domain knowledge.

1.5 Proposed System

PromptForge proposes a "system-of-systems" approach where a swarm of specialized AI agents collaborates to perform prompt engineering automatically. The system is architected in distinct layers: a presentation layer (API), a business logic layer (Service), and an AI orchestration layer (Workflow). It uses a fork-join pattern to execute multiple analysis agents in parallel, significantly reducing latency. The user provides a simple input, and the system handles the complex task of adding detail, structure, and constraints, ultimately producing a prompt that is orders of magnitude more detailed than the original.

2. Literature Survey

[This section requires academic and industry research on topics such as prompt engineering, multi-agent systems (MAS), graph-based LLM orchestration (e.g., LangGraph), and parallel execution models in AI. This analysis is beyond the scope of automated code review and should be completed by the project authors. Key areas to research would include papers on agentic workflows, state graph management in LLM applications, and performance comparisons of different agent architectures.]

---

3. System Requirements and Specification

3.1 General Description of the System

PromptForge is a multi-tenant, API-driven service. The system is designed to be stateless at the application layer, with all state managed externally in the database (Supabase) and cache (Redis). The core workflow is asynchronous and event-driven, with capabilities for both standard request/response and real-time streaming via Server-Sent Events (SSE).

3.2 Technical Requirements of the System

Table 1: System Software Requirements

Category	Requirement	Description
Operating System	Ubuntu 22.04 LTS / Windows 11	Recommended for development and deployment stability.
Programming Language	Python 3.11 +	Main language for backend logic and ML integration.
Web Framework	FastAPI	High-performance ASGI framework for building the RESTful API.
AI Orchestration	LangGraph, LangChain	For building the stateful, multi-agent graph workflow.
Database	Supabase (PostgreSQL + pgvector)	For storing user data, session history, and vector embeddings for memory.
Cache	Redis	For low-latency caching of swarm results to improve performance.
Observability	Sentry, Langsmith, OpenTelemetry	For error tracking, LLM tracing, and infrastructure monitoring.
API Security	python-jose[cryptography]	For JWT token validation and authentication.
Deployment	Docker, Uvicorn	For containerization and serving the ASGI application.

Table 2: Hardware Specifications

Component	Minimum Specification	Recommended Specification
Processor	Quad-core (Intel i5 / AMD Ryzen 5)	Hexa-core or higher (Intel i7 / Ryzen 7)

Memory (RAM)	8 GB	16 GB (for handling multiple concurrent requests)
Storage	256 GB SSD	512 GB+ NVMe SSD (for fast data I/O and logging)
Connectivity	5 Mbps Internet	50 Mbps+ (for low-latency communication with external APIs)

4. System Design and Analysis

4.1 System Architecture

The architecture of PromptForge is a modular, three-tier design that separates presentation, business logic, and AI orchestration. This promotes maintainability, scalability, and testability.

1. **API Layer (`api.py`):** The system's entrypoint, built with FastAPI. This layer is responsible for request handling, authentication, and middleware execution. It uses a factory pattern to construct the app and register all API routers. Key middleware includes:

    - `CORSMiddleware`: Restricts access to allowed frontend domains.
    - `RateLimiterMiddleware`: Prevents abuse by limiting requests per user.
    - `MetricsMiddleware`: Tracks latency and other performance metrics.
    - `Sentry & OpenTelemetry`: Provide robust error tracking and distributed tracing.

2. **Service Layer (`service.py`):** This layer contains the core business logic and is decoupled from the web framework. Its primary responsibilities include:

    - **Workflow Execution:** The `_run_swarm` function acts as the main controller, preparing the initial state for the AI workflow.
    - **Caching Logic:** It interfaces with Redis (`get_cached_result`, `set_cached_result`) to cache the expensive results of the agent swarm, providing near-instantaneous responses for repeated queries.
    - **State Management:** It initializes the `PromptForgeState` object, a comprehensive data structure that carries information through the entire AI workflow.
    - **Utility Functions:** Provides helpers like `compute_diff` for frontend display and `sse_format` for streaming responses.

3. **AI Orchestration Layer (`workflow.py`):** This is the heart of the AI, built using LangGraph. It defines a `StateGraph` that manages the flow of data between different AI agents in a "fork-join" pattern.

    - **Entrypoint (`kira_orchestrator`):** A lightweight LLM call that analyzes the user's request and decides which specialized agents need to be activated.
    - **Parallel Execution (`Send`):** The system uses LangGraph's `Send` API to dispatch tasks to the `intent_agent`, `context_agent`, and `domain_agent` simultaneously. This true parallelism is a critical performance optimization.
    - **Join Node (`prompt_engineer`):** This agent waits for all parallel agents to complete their analysis. It then synthesizes their outputs into the final, engineered prompt. This ensures the final output is holistic and well-informed.

```
+--------------------------+      +------------------------+
|   User (via Frontend)    |----->|   API Layer (api.py)   |
+--------------------------+      | - FastAPI              |
                                  | - Middleware (Auth,    |
                                  |   Rate Limit, Sentry)  |
                                  +-----------+------------+
                                              |
                                              v
+--------------------------+      +-----------+------------+
| Cache (Redis)            |<---->| Service Layer (service.py)|
+--------------------------+      | - Business Logic       |
                                  | - Caching              |
                                  | - State Initialization |
                                  +-----------+------------+
                                              |
                                              v
+-------------------------------------------------------------------+
| AI Orchestration Layer (workflow.py)                              |
| +--------------------+  +--------------------+  +-----------------+ |
| |   Intent Agent     |  |   Context Agent    |  |  Domain Agent   | |
| +--------------------+  +--------------------+  +-----------------+ |
|          |                     |                     |            |
|          +----------------------+---------------------+            |
|                                |                                   |
|                                v                                   |
|                      +---------------------+                      |
|                      | Prompt Engineer Agent |  <-- Join Node      |
|                      +---------------------+                      |
+-------------------------------------------------------------------+
```

4.2 Module Description & Pseudocode

Module 1: Swarm Execution Service (`_run_swarm`)

This function is the primary entrypoint into the business logic. It orchestrates the cache check, state creation, and workflow invocation.

Pseudocode for Swarm Execution:

```
FUNCTION _run_swarm(prompt, user_id):
    # 1. Check for a cached result to avoid expensive AI calls
    cached_result = get_cached_result(prompt, user_id)
    IF cached_result IS NOT NULL:
        RETURN cached_result

    # 2. Initialize the state object for the workflow
    initial_state = PromptForgeState(
        message=prompt,
        user_id=user_id,
        # ... other fields initialized to default
    )

    # 3. Invoke the LangGraph workflow in a thread pool for timeout han
```

```
    WITH ThreadPoolExecutor(timeout=180) as executor:
        future = executor.submit(workflow.invoke, initial_state)
        result = future.result()

    # 4. Cache the new result for future requests
    set_cached_result(prompt, result, user_id)

    # 5. Trace the execution for observability
    trace_swarm_run(result)

    RETURN result
```

Module 2: AI Workflow Graph (`workflow.py`)

This module defines the state machine for the agent swarm using LangGraph.

Pseudocode for Graph Definition:

```
FUNCTION build_graph():
    graph = StateGraph(AgentState)

    # 1. Define all nodes in the graph
    graph.add_node("kira_orchestrator", orchestrator_node)
    graph.add_node("intent_agent", intent_agent)
    graph.add_node("context_agent", context_agent)
    graph.add_node("domain_agent", domain_agent)
    graph.add_node("prompt_engineer", prompt_engineer_agent)

    # 2. Set the entry point
    graph.set_entry_point("kira_orchestrator")

    # 3. Define conditional routing from the orchestrator
    # This function returns Send() objects for parallel execution
    FUNCTION route_to_agents(state):
        decision = state.orchestrator_decision
        agents_to_run = decision.agents_to_run
        RETURN [Send(agent) for agent in agents_to_run]

    graph.add_conditional_edges("kira_orchestrator", route_to_agents)

    # 4. Define edges from parallel agents to the join node
    graph.add_edge("intent_agent", "prompt_engineer")
    graph.add_edge("context_agent", "prompt_engineer")
    graph.add_edge("domain_agent", "prompt_engineer")

    # 5. Define the exit point
    graph.add_edge("prompt_engineer", END)

    RETURN graph.compile()
```

---

5. Conclusion and Future Enhancement

5.1 Conclusion

PromptForge v2.0 is a robust, production-ready implementation of a multi-agent AI system for automated prompt engineering. Its modular, three-tier architecture ensures separation of concerns and high maintainability. The strategic use of caching, parallel execution, and comprehensive observability tooling demonstrates a mature approach to building scalable AI services. By successfully abstracting the complexity of prompt refinement into an automated workflow, the project provides significant value by improving both the quality of AI interactions and the efficiency of the users.

5.2 Future Directions and Enhancement

- **Dynamic Agent Loading:** The agent swarm is currently static. A future version could dynamically load or select agents from a larger pool based on the specific domain of the user's prompt, allowing for greater specialization.
- **Self-Healing Workflows:** The graph could be enhanced with retry logic and fallback mechanisms within the `StateGraph` itself, allowing it to recover from transient LLM or API failures without manual intervention.
- **Advanced Memory:** The current memory system could be evolved into a true long-term memory module using techniques like BM25 for hybrid search, allowing the system to recall information from a much larger context window.
- **Cost-Benefit Analysis:** The orchestrator could be made more sophisticated by performing a cost-benefit analysis, choosing between faster, cheaper models and slower, more powerful models based on the user's request complexity or subscription tier.

---

6. References

[This section requires manual completion. Based on the codebase, key references would include the official documentation for FastAPI, LangChain, LangGraph, Sentry, and the papers describing the underlying LLM models used (e.g., Qwen models from Alibaba Cloud).]

---

7. Appendix

Appendix A: API Middleware Stack

The `api.py` file configures the following middleware in order of execution:

1. **Sentry Integration:** (Initialized first) Captures errors and performance data across the entire application lifecycle.
2. **CORS Middleware:** Enforces Cross-Origin Resource Sharing policies to ensure requests only come from authorized frontend applications.
3. **Rate Limiter Middleware:** Protects the API from denial-of-service attacks and abuse by enforcing a request limit (e.g., 100 requests/hour) per user.
4. **Metrics Middleware:** Injects structured logging and calculates request/response latency for monitoring and performance analysis.
5. **OpenTelemetry Middleware:** (Optional) Provides distributed tracing for deep infrastructure-level observability.

Appendix B: `PromptForgeState` Data Dictionary

The `PromptForgeState` is a Pydantic model that serves as the central data object for the LangGraph workflow. Key fields include:

Field	Type	Description
`message`	str	The user's current input prompt.
`user_id`	str	The authenticated user's unique identifier.
`session_id`	str	The identifier for the current conversation.
`orchestrator_decision`	dict	The output from the first routing agent.
`intent_analysis`	dict	The output from the intent agent.
`context_analysis`	dict	The output from the context agent.
`domain_analysis`	dict	The output from the domain agent.
`improved_prompt`	str	The final, engineered prompt from the `prompt_engineer` agent.
`quality_score`	dict	A structured score evaluating the quality of the final prompt.
`agents_run`	List[str]	A list of agents that were executed in the workflow.
`agent_latencies`	dict	A dictionary tracking the execution time of each agent.